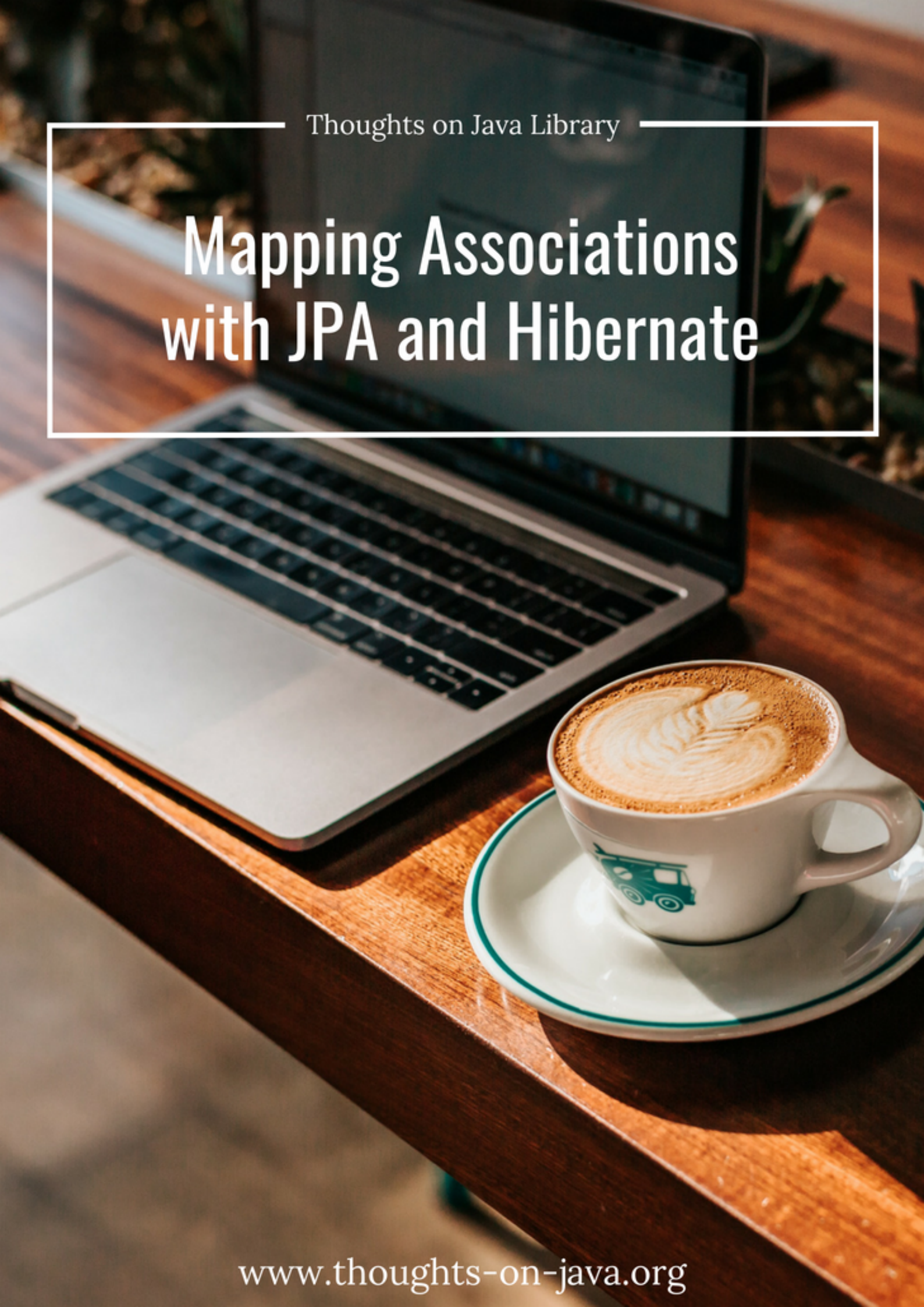
Thoughts on Java Library

Mapping Associations with JPA and Hibernate



www.thoughts-on-java.org

Mapping Associations with JPA and Hibernate

Thorben Janssen

Mapping Associations with JPA and Hibernate

© 2018 Thorben Janssen. All rights reserved.

Thorben Janssen
Elser Bruch 63
33106 Paderborn
Germany
http://www.thoughts-on-java.org

All rights reserved. No part of this book may be reproduced, stored in a retrieval system or transmitted in any form or by any means, without the prior written permission of the author, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. The author will not be held liable for any damages caused or alleged to be caused directly or indirectly by this book.

Table of Contents

Preface . . . 1

Mapping Associations . . . 2

- Many-to-One Associations . . . 3
- Many-to-Many Associations . . . 9
- One-to-One Associations . . . 14

The Most Efficient Data Type for To-Many Associations . . . 17

- The difference between a Bag and a List . . . 18
- Should you use a Bag or a Set? . . . 19

Mapping an association as a java.util.Map . . . 23

- Mapping simple key-value pairs . . . 24
- Mapping multiple values to the same key . . . 29

Introducing JPA FetchTypes . . . 33

- Default FetchTypes and how to change them . . . 35
- `FetchType.EAGER` - Fetch it so you'll have it when you need it . . . 37
- `FetchType.LAZY` - Fetch it when you need it . . . 39

Thorben Janssen . . . 42

Preface

Association mappings are one of the key features of JPA and Hibernate [http://www.hibernate.org]. They model the relationship between two database tables as entity attributes. That allows you to easily navigate them in your domain model and JPQL [https://www.thoughts-on-java.org/jpql/] or Criteria queries.

JPA and Hibernate support the same associations as you know from your relational database model. Their mapping might seem easy. However, if you ever struggled with performance issues or tried to define a more complex mapping, you know that a good mapping requires more than just a simple annotation. You need a solid understanding of the underlying concepts. That's especially the case for association mappings.

That's why I collected some of my most important articles about association mappings in this ebook. In the following chapters, I will show you:

- How you can map the different types of associations.
- How to choose the optimal data type for your to-many associations.
- How to get index-based access to the elements of a to-many association by mapping it to a `java.util.Map`.
- How to define when your JPA implementation fetches associated entities from the database.

Mapping Associations

JPA and Hibernate support the same kinds of associations as you use in your table model. You can map:

- one-to-one associations,
- many-to-one associations and
- many-to-many associations.

You can map each of them as a uni- or bi-directional association. That means you can either model them as an attribute on only one of the associated entities or on both. That has no impact on your database mapping, but it defines in which direction you can use the relationship in your domain model and JPQL or Criteria queries. I will explain that in more details in the first example.

Many-to-One Associations

An order consists of multiple items, but each item belongs to only one order. That is a typical example for a many-to-one association. If you want to model this in your database model, you need to store the primary key [https://www.thoughts-on-java.org/jpa-generate-primary-keys/] of the `Order` record as a foreign key in the `OrderItem` table. With JPA and Hibernate, you can model this in 3 different ways. You can either model it as a bidirectional association with an attribute on the `Order` and the `OrderItem` entity. Or you can model it as a unidirectional relationship with an attribute on the `Order` or the `OrderItem` entity.

Unidirectional Many-to-One Association

Let's take a look at the unidirectional mapping on the `OrderItem` entity first. The `OrderItem` entity represents the many side of the relationship and the `OrderItem` table contains the foreign key of the record in the `Order` table. As you can see in the following code snippet, you can model this association with an attribute of type `Order` and a `@ManyToOne` annotation. The `Order order` attribute models the association, and the annotation tells Hibernate how to map it to the database.

```
@Entity
public class OrderItem {

    @ManyToOne
    private Order order;

    ...
}
```

That is all you need to do to model this association. By default, Hibernate generates the name of the foreign key column based on the name of the relationship mapping attribute and the name of the primary key attribute. In this example, Hibernate would use a column with the name `order_id` to store the foreign key to the `Order` entity.

If you want to use a different column, you need to define the foreign key column name

with a `@JoinColumn` annotation. The example in the following code snippet tells Hibernate to use the column `fk_order` to store the foreign key.

```
@Entity
public class OrderItem {

    @ManyToOne
    @JoinColumn(name = "fk_order")
    private Order order;

    ...
}
```

You can now use this association in your business code to get the `Order` for a given `OrderItem` and to add or remove an `OrderItem` to or from an existing `Order`.

```
Order o = em.find(Order.class, 1L);

OrderItem i = new OrderItem();
i.setOrder(o);

em.persist(i);
```

That's all about the mapping of unidirectional many-to-one associations for now. If you want to dive deeper, you should take a look at `FetchTypes`. I explain them in more details in the Introducing JPA FetchTypes chapter.

But now, let's continue with the association mappings and talk about unidirectional one-to-many relationships next. As you might expect, the mapping is very similar to this one.

Unidirectional One-to-Many Association

The unidirectional one-to-many relationship mapping is not very common. In the example of this book, it only models the association on the `Order` entity and not on the `OrderItem`. The basic mapping definition is very similar to the many-to-one association. It consist of the `List items` attribute which stores the associated entities and a `@OneToMany`

association.

```
@Entity
public class Order {

    @OneToMany
    private List<OrderItem> items = new ArrayList<OrderItem>();

    ...
}
```

But this is most likely not the mapping you're looking for because Hibernate uses an association table to map the relationship. If you want to avoid that, you need to use a `@JoinColumn` annotation to specify the foreign key column.

The following code snippet shows an example of such a mapping. The `@JoinColumn` annotation tells Hibernate to use the `fk_order` column in the `OrderItem` table to join the two database tables.

```
@Entity
public class Order {

    @OneToMany
    @JoinColumn(name = "fk_order")
    private List<OrderItem> items = new ArrayList<OrderItem>();

    ...
}
```

You can now use the association in your business code to get all `OrderItem` s of an `Order` and to add or remove an `OrderItem` to or from an `Order`.

```
Order o = em.find(Order.class, 1L);

OrderItem i = new OrderItem();

o.getItems().add(i);

em.persist(i);
```

Bidirectional Many-to-One Associations

The bidirectional Many-to-One association mapping is the most common way to model this relationship with JPA and Hibernate. It uses an attribute on the `Order` and the `OrderItem` entity. This allows you to navigate the association in both directions in your domain model and your JPQL queries [https://www.thoughts-on-java.org/jpql/]. The mapping definition consists of 2 parts:

- the to-many side of the association which owns the relationship mapping and
- the to-one side which just references the mapping

Let's take a look at the owning side first. You already know this mapping from the unidirectional Many-to-One association mapping. It consists of the `Order order` attribute, a `@ManyToOne` annotation and an optional `@JoinColumn` annotation.

```
@Entity
public class OrderItem {

    @ManyToOne
    @JoinColumn(name = "fk_order")
    private Order order;

    ...
}
```

The owning part of the association mapping already provides all the information Hibernate needs to map it to the database. That makes the definition of the referencing part simple. You just need to reference the owning association mapping. You can do that

by providing the name of the association-mapping attribute to the `mappedBy` attribute of the `@OneToMany` annotation. In this example, that's the `order` attribute of the `OrderItem` entity.

```
@Entity
public class Order {

    @OneToMany(mappedBy = "order")
    private List<OrderItem> items = new ArrayList<OrderItem>();

    ...
}
```

You can now use this association in a similar way as the unidirectional relationships I showed you before. But adding and removing an entity from the relationship requires an additional step. You need to update both sides of the association.

```
Order o = em.find(Order.class, 1L);

OrderItem i = new OrderItem();
i.setOrder(o);

o.getItems().add(i);

em.persist(i);
```

That is an error-prone task, and a lot of developers prefer to implement it in a utility method which updates both entities.

```
@Entity
public class Order {
    ...

    public void addItem(OrderItem item) {
        this.items.add(item);
        item.setOrder(this);
    }
    ...
}
```

That's all about many-to-one association mapping for now. You should also take a look at `FetchTypes` and how they impact the way Hibernate loads entities from the database. I get into detail about them in the Introducing JPA FetchTypes chapter.

Many-to-Many Associations

Many-to-Many relationships are another often used association type. On the database level, it requires an additional association table which contains the primary key [https://www.thoughts-on-java.org/jpa-generate-primary-keys/] pairs of the associated entities. But as you will see, you don't need to map this table to an entity.

A typical example for such a many-to-many association are `Product` s and `Store` s. Each `Store` sells multiple `Product` s and each `Product` gets sold in multiple `Store` s. Similar to the many-to-one association, you can model a many-to-many relationship as a uni- or bidirectional relationship between two entities. But there is an important difference that might not be obvious when you look at the following code snippets. When you map a many-to-many association, you should use a `Set` instead of a `List` as the attribute type. Otherwise, Hibernate will take a very inefficient approach to remove entities from the association. It will remove all records from the association table and re-insert the remaining ones. You can avoid that by using a `Set` instead of a `List` as the attribute type.

OK let's take a look at the unidirectional mapping first.

Unidirectional Many-to-Many Associations

Similar to the previously discussed mappings, the unidirectional many-to-many relationship mapping requires an entity attribute and a `@ManyToMany` annotation. The attribute models the association and you can use it to navigate it in your domain model or JPQL queries [https://www.thoughts-on-java.org/jpql/]. The annotation tells Hibernate to map a many-to-many association.

Let's take a look at the relationship mapping between a `Store` and a `Product`. The `Set products` attribute models the association in the domain model and the `@ManyToMany` association tells Hibernate to map it as a many-to-many association. And as I already explained, please note the difference to the previous many-to-one mappings. You should map the associated entities to a `Set` instead of a `List.`

```
@Entity
public class Store {

    @ManyToMany
    private Set<Product> products = new HashSet<Product>();

    ...
}
```

If you don't provide any additional information, Hibernate uses its default mapping which expects an association table with the name of both entities and the primary key attributes of both entities. In this case, Hibernate uses the `Store_Product` table with the columns `store_id` and `product_id`.

You can customize that with a `@JoinTable` annotation and its attributes `joinColumns` and `inverseJoinColumns`. The `joinColumns` attribute defines the foreign key columns for the entity on which you define the association mapping. The `inverseJoinColumns` attribute specifies the foreign key columns of the associated entity.

The following code snippet shows a mapping that tells Hibernate to use the `store_product` table with the `fk_product` column as the foreign key to the `Product` table and the `fk_store` column as the foreign key to the `Store` table.

```
@Entity
public class Store {

    @ManyToMany
    @JoinTable(name = "store_product",
           joinColumns = { @JoinColumn(name = "fk_store") },
           inverseJoinColumns = { @JoinColumn(name = "fk_product") })
    private Set<Product> products = new HashSet<Product>();

    ...
}
```

That's all you have to do to define an unidirectional many-to-many association between two entities. You can now use it to get a `Set` of associated entities in your domain model

or to join the mapped tables in a JPQL query.

```
Store s = em.find(Store.class, 1L);

Product p = new Product();

s.getProducts().add(p);

em.persist(p);
```

Bidirectional Many-to-Many Associations

The bidirectional relationship mapping allows you to navigate the association in both directions. And after you've read this far, you're probably not surprised when I tell you that the mapping follows the same concept as the bidirectional mapping of a many-to-one relationship.

One of the two entities owns the association and provides all mapping information. The other entity just references the association mapping so that Hibernate knows where it can get the required information.

Let's start with the entity that owns the relationship. The mapping is identical to the unidirectional many-to-many association mapping. You need an attribute that maps the association in your domain model and a `@ManyToMany` association. If you want to adapt the default mapping, you can do that with a `@JoinColumn` annotation.

```
@Entity
public class Store {

    @ManyToMany
    @JoinTable(name = "store_product",
           joinColumns = { @JoinColumn(name = "fk_store") },
           inverseJoinColumns = { @JoinColumn(name = "fk_product") })
    private Set<Product> products = new HashSet<Product>();

    ...
}
```

The mapping for the referencing side of the relationship is a lot easier. Similar to the bidirectional many-to-one relationship mapping, you just need to reference the attribute that owns the association.

You can see an example of such a mapping in the following code snippet. The `List products` attribute of the `Store` entity owns the association. So, you only need to provide the `String` "products" to the `mappedBy` attribute of the `@ManyToMany` annotation.

```
@Entity
public class Product{

    @ManyToMany(mappedBy="products")
    private Set<Store> stores = new HashSet<Store>();

    ...
}
```

That's all you need to do to define a bidirectional many-to-many association between two entities. But there is another thing you should do to make it easier to use the bidirectional relationship. You need to update both ends of a bidirectional association when you want to add or remove an entity. Doing that in your business code is verbose and error-prone. It's, therefore, a good practice to provide helper methods which update the associated entities.

```java
@Entity
public class Store {

    public void addProduct(Product p) {
        this.products.add(p);
        p.getStores().add(this);
    }

    public void removeProduct(Product p) {
        this.products.remove(p);
        p.getStores().remove(this);
    }

    ...
}
```

OK, now we're done with the definition of the many-to-many association mappings. Let's take a look at the third and final kind of association: The one-to-one relationship.

One-to-One Associations

One-to-one relationships are rarely used in relational table models. You, therefore, won't need this mapping too often. But you will run into it from time to time. So it's good to know that you can map it in a similar way as all the other associations. An example for a one-to-one association could be a `Customer` and the `ShippingAddress`. Each `Customer` has exactly one `ShippingAddress` and each `ShippingAddress` belongs to one `Customer`. On the database level, this gets mapped by a foreign key column either on the `ShippingAddress` or the `Customer` table. Let's take a look at the unidirectional mapping first.

Unidirectional One-to-One Associations

As in the previous unidirectional mapping, you only need to model it for the entity for which you want to navigate the relationship in your query or domain model. Let's say you want to get from the `Customer` to the `ShippingAddress` entity.

The required mapping is similar to the previously discussed mappings. You need an entity attribute that represents the association, and you have to annotate it with an `@OneToOne` annotation. When you do that, Hibernate uses the name of the associated entity and the name of its primary key attribute to generate the name of the foreign key column. In this example, it would use the column `shippingaddress_id`. You can customize the name of the foreign key column with a `@JoinColumn` annotation. The following code snippet shows an example of such a mapping.

```
@Entity
public class Customer{

    @OneToOne
    @JoinColumn(name = "fk_shippingaddress")
    private ShippingAddress shippingAddress;

    ...
}
```

That's all you need to do to define a one-to-one association mapping. You can now use it

in your business code to add or remove an association, to navigate it in your domain model or to join it in a JPQL query.

```
Customer c = em.find(Customer.class, 1L);
ShippingAddress sa = c.getShippingAddress();
```

Bidirectional One-to-One Associations

The bidirectional one-to-one relationship mapping extends the unidirectional mapping so that you can also navigate it in the other direction.

In this example, you also model it on the `ShippingAddress` entity so that you can get the `Customer` for a giving `ShippingAddress`. Similar to the previously discussed bidirectional mappings, the bidirectional one-to-one relationship consists of an owning and a referencing side. The owning side of the association defines the mapping, and the referencing one just links to that mapping.

The definition of the owning side of the mapping is identical to the unidirectional mapping. It consists of an attribute that models the relationship and is annotated with a `@OneToOne` annotation and an optional `@JoinColumn` annotation.

```
@Entity
public class Customer{

    @OneToOne
    @JoinColumn(name = "fk_shippingaddress")
    private ShippingAddress shippingAddress;

    ...
}
```

The referencing side of the association just links to the attribute that owns the relationship. Hibernate gets all information from the referenced mapping, and you don't need to provide any additional information. You can define that with the `mappedBy` attribute of the `@OneToOne` annotation. The following code snippet shows an example of

such a mapping.

```
@Entity
public class ShippingAddress{

    @OneToOne(mappedBy = "shippingAddress")
    private Customer customer;

    ...
}
```

The Most Efficient Data Type for To-Many Associations

In the previous chapter, I shortly mentioned that it's important to choose the correct data type to model your to-many association. Let's dive a little deeper into that topic.

So, which data type should you use to map a to-many association with Hibernate? Is it better to use a `Set` or a `List`? That's a very common question, and most developers are surprised when they look at the documentation and find out that these are not the only options. You can also use a `Bag` or a `Map`.

The mapping to a `Map` is a special case, which I will explain it in great details in one of the following chapters. In contrast to the other 3 options, it provides indexed based access to the associated entities. That might be beneficial for some use cases, but the creation and management of the `Map` also creates an overhead. It should, therefore, not be your default choice.

So, let's focus on the other 3 options. `List`, `Set`, and `Bag` represent simple collections of entities. The first 2 options should sound familiar because the `java.util` package contains a `List` and a `Set` interface. But what is a `Bag`? There is no class in the `java.util` package with that name.

The difference between a Bag and a List

Hibernate's naming of the different collection types is a little bit confusing because `List` s and `Bag` s are both mapped by a `java.util.List`. The difference between them is that a `List` is ordered and a `Bag` is unordered.

If you map your to-many association to a `java.util.List` without specifying the order of your association elements [http://www.thoughts-on-java.org/ordering-vs-sorting-hibernate-use/], you're using a `Bag` and not a List. That should be the case for most of your associations because retrieving the association in a specific order slows down your database queries. You should better use a JPQL query [http://www.thoughts-on-java.org/jpql/] with an ORDER BY clause to define the ordering if you need it.

So, for most association mappings, there are 2 options remaining. You need to decide between a `Bag` and a `Set`.

Should you use a Bag or a Set?

When you just look at the Java types, the answer seems to be easy. In general, a `java.util.List` provides the better performance while a `java.util.Set` doesn't contain any duplicates.

As long as you implement the create use case correctly, a `java.util.List` seems like the obvious best choice for your association mapping. But it's not that easy. A `List` might be more efficient than a `Set`, but the type also influences how Hibernate manages the association in the database. So, there are a few other things you need to take into account when you make your decision.

A critical bug in older Hibernate versions

First of all, if you're using a Hibernate version older than 5.0.8, you should be aware of bug HHH-5855 [https://hibernate.atlassian.net/browse/HHH-5855]. When you used a `java.util.List` and merged the parent entity [http://www.thoughts-on-java.org/persist-save-merge-saveorupdate-whats-difference-one-use/], Hibernate generated 2 INSERT statements for each new child entity.

But that's not the only thing you need to worry about.

Inefficient handling of many-to-many associations

If you model the association as a `java.util.List`, Hibernate handles the removal of associated entities very inefficiently.

```
@Entity
public class Book {

    // DON'T DO THIS!!!
    @ManyToMany
    @JoinTable(name = "book_author",
            joinColumns = { @JoinColumn(name = "fk_book") },
            inverseJoinColumns = { @JoinColumn(name = "fk_author") })
    private List authors = new ArrayList();

    ...
}
```

In the following code snippet, I load a `Book` which was written by 2 `Author` s and remove one of the `Author` s from the association.

```
em = emf.createEntityManager();
em.getTransaction().begin();

// Get Book entity with 2 Authors
b = em.find(Book.class, 1L);

// Remove one of the Author
b.getAuthors().remove(a);

em.getTransaction().commit();
em.close();
```

As you can see in the log messages, Hibernate removes all records from the association table before it inserts a new record for the remaining association. This approach is obviously very inefficient. Depending on the number of associated entities, the additional INSERT statements can create severe performance problems.

```
...
09:54:28,876 DEBUG [org.hibernate.SQL] - update Book set title=?, version=?
where id=? and version=?
09:54:28,878 DEBUG [org.hibernate.SQL] - delete from book_author where
fk_book=?
09:54:28,882 DEBUG [org.hibernate.SQL] - insert into book_author (fk_book,
fk_author) values (?, ?)
```

Use a Set to map many-to-many associations

Hibernate handles the association a lot better if you model it as a `java.util.Set`.

```
@Entity
public class Book {

    @ManyToMany
    @JoinTable(name = "book_author",
            joinColumns = { @JoinColumn(name = "fk_book") },
            inverseJoinColumns = { @JoinColumn(name = "fk_author") })
    private Set authors = new HashSet();

    ...
}
```

If you run the same test case again, Hibernate now only removes the record that represents the removed association. As expected, all other database records are not affected by the remove operation.

```
...
10:00:37,709 DEBUG [org.hibernate.SQL] - update Book set title=?, version=?
where id=? and version=?
10:00:37,711 DEBUG [org.hibernate.SQL] - delete from book_author where
fk_book=? and fk_author=?
```

As you've seen, mapping an association as a `java.util.List` can create problems which by far outweigh the small performance gain you get compared to a `java.util.Set`. You should make sure to update your Hibernate version and to use a `Set` to model many-to-

many associations.

Mapping an association as a java.util.Map

The `java.util.List<SomeEntityClass>` and `java.util.Set<SomeEntityClass>` are the most common representation of a to-many association with JPA and Hibernate. But are they also the ones that you want to use in your domain model? Do they fit your use cases or do you need smarter access to the associated elements?

Let's be honest, a simple `java.util.Set` is good enough in most cases. But from time to time, a `java.util.Map` would make the implementation of your business logic so much easier. So, why not use JPA's and Hibernate's mapping capabilities and map the association to a `Map`? Only for the relationships that are used as `Map` s, of course.

The mapping definition requires just a few additional annotations.

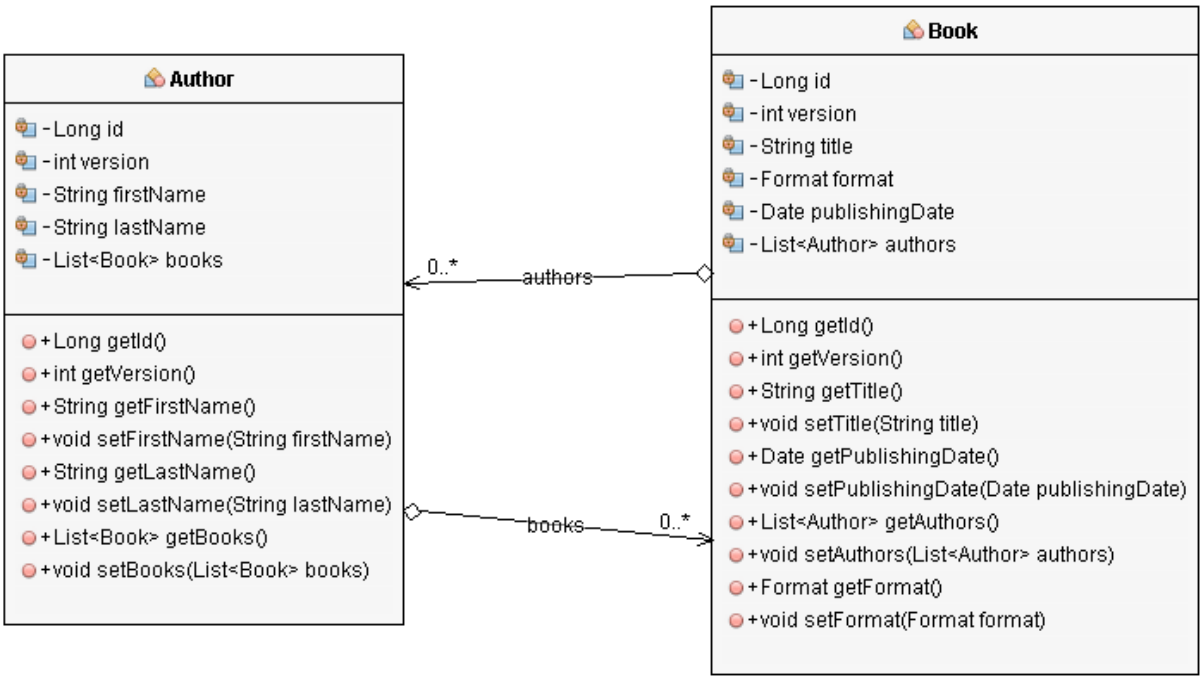
Mapping simple key-value pairs

The basic mappings are pretty simple. JPA and Hibernate allow you to map a `Collection` of elements or an entity association to a `java.util.Map`. The key and value of the `Map` can be a basic type, an embeddable class or an entity. You might have recognized that `Collection` s are not supported. If you want to map your association to a `Map<YourKeyClass, List<YourValueClass>>`, you need to use a small workaround that I show you at the end of this chapter.

But let's take a look at some simple mapping first and discuss the mapping concept.

Represent a to-Many Association as a `Map<String, EntityClass>`

Let's start with a standard mapping definition of a many-to-many association that uses a `List`. The following diagram shows an example of a many-to-many association between the `Author` and the `Book`.



And you can see the standard mapping using a `List` in the following code snippet.

```
@Entity
public class Author {

    @ManyToMany
    @JoinTable(
        name="BookAuthor",
        joinColumns={@JoinColumn(name="bookId", referencedColumnName="id")},
        inverseJoinColumns={@JoinColumn(name="authorId",
referencedColumnName="id")})
    private List<Book> books = new ArrayList<Book>();

    ...
}
```

If you want to represent the association with a `Map` instead of the `List`, you just have to do 2 things:

1. change the type of the attribute from `List` to `Map` and
2. define the database column you want to use as a map key.

Let's say you want to use the `title` column of the `Book` table as the map key. The title is a `String` attribute of the `Book`. So, it's a basic type that's already mapped by your entities and you just need to reference it in a `@MapKey` annotation. If you don't want to use one of the attributes of the associated entity, you can also use:

- a basic type with a `@MapKeyColumn` annotation
- an enum with a `@MapKeyEnumerated` annotation
- a `java.util.Date` or a `java.util.Calendar` with a `@MapKeyTemporal` annotation

Hibernate will persist the map key in an additional database column.

But back to our example. When you apply the described changes, the mapping of the `Author` entity looks like this.

```
@Entity
public class Author {

    @ManyToMany
    @JoinTable(
        name="AuthorBookGroup",
        joinColumns={@JoinColumn(name="fk_author", referencedColumnName=
"id")},
        inverseJoinColumns={@JoinColumn(name="fk_group",
referencedColumnName="id")})
    @MapKey(name = "title")
    private Map<String, Book> books = new HashMap<String, Book>();

    ...
}
```

Working with `Map` s instead of `List` s

That's all you need to do to define the mapping. You can now use the map in your domain model to add, retrieve or remove elements from the association.

```
Author a = new Author();
a.setFirstName("Thorben");
a.setLastName("Janssen");
em.persist(a);

Book b = new Book();
b.setTitle("Hibernate Tips");
b.setFormat(Format.PAPERBACK);
b.getAuthors().add(a);
em.persist(b);

a.getBooks().put(b.getTitle(), b);
```

You also have 2 options to use the association in a JPQL query. You can simply join the 2 entities and use them as a regular many-to-many relationship.

```
TypedQuery<Author> q = em.createQuery("SELECT a FROM Author a JOIN a.books b
WHERE b.title LIKE :title", Author.class);
q.setParameter("title", "%Hibernate Tips%");
a = q.getSingleResult();
```

Or you can use the `KEY` function to reference the map key in your query.

```
TypedQuery<Author> q = em.createQuery("SELECT a FROM Author a JOIN a.books b
WHERE KEY(b) LIKE :title ", Author.class);
q.setParameter("title", "%Hibernate Tips%");
a = q.getSingleResult();
```

Represent a `Collection` of Embeddables as a `Map<EnumClass, EntityClass>`

The mapping for embeddable classes is pretty similar. The main difference is that you need to use the `@ElementCollection` annotation instead of a `@ManyToMany` annotation. Let's say you want to persist an author with her business and private address. In your domain model, you might model that with the 3 classes you can see in the following diagram. image::Author-Address-AddressType.png[Author-Address-AddressType,width=622,height=431]] The `Author` class becomes an entity and you can implement the `Address` class as an embedabble. And for this example, I don't want to keep the `Address`es in a simple `ElementCollection`. I want to use a `Map` with the `AddressType` enum. As in the previous example, you only need to do 2 things to use a `Map` instead of a `List`:

1. change the type of the attribute from `List` to `Map`
2. define the mapping of the map key

The second step even becomes optional if you only use the enum as a map key and not as an attribute of the embeddable. If you don't provide any additional information, Hibernate uses a set of defaults to persist the map key in a column of the collection table. I prefer to define the column name and the `EnumType` Hibernate shall use to persist the enum.

You can see an example for that in the following code snippet. The `name` attribute of the `@MapKeyColumn` annotation defines the name of the database column and the `@MapKeyEnumerated` defines how Hibernate shall persist the enum [https://www.thoughts-on-java.org/hibernate-tips-map-enum-database-column/].

```
@Entity
public class Author {

    @ElementCollection
    @MapKeyColumn(name = "address_type")
    @MapKeyEnumerated(EnumType.STRING)
    private Map<AddressType, Address>address = new HashMap<AddressType,
Address>();

    ...
}
```

That's all you have to do to represent a collection of embeddables as a `java.util.Map`. You can now use it in the same way as I showed you in the previous example.

Mapping multiple values to the same key

As you've seen in the previous examples, JPA and Hibernate don't support any `Collection` type as a map value. That's unfortunate because, in my experience, that is the most common use case. But there is a small and easy workaround for these situations. You just need to introduce a wrapper entity that wraps the `List` or `Map` of entities.

Let's extend the example of the `Author`-`Book`-association I showed you at the beginning of this chapter.

Most `Book` s get published in different `Format` s, e.g. as an ebook and as a paperback. The `Book` has a unique ISBN for each `Format`. So, you need 2 database records to persist the ebook and paperback version of a `Book`. But you can't persist these 2 records with the mapping from our first example. The 2nd `Book` entity would replace the 1st one, when you add it to the `Map<String, Book> books`. You can fix that by introducing an additional entity that wraps the `List` of `Book` entities. image::BookGroup.png[BookGroup,width=1195,height=432]] The mapping definition of that entity is very simple. You just need a primary key [https://www.thoughts-on-java.org/jpa-generate-primary-keys/] and a one-to-many association to the `Book` entity. I also introduced a `title` for the `BookGroup` which I want to use as the map key on the `Author` entity.

```
@Entity
public class BookGroup {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name = "id", updatable = false, nullable = false)
    private Long id;

    @OneToMany(mappedBy = "group")
    private List<Book> books = new ArrayList<Book>();

    private String title;

    ...
}
```

You can then model the association between the `BookGroup` and the `Author` entity as a `Map`.

```
@Entity
public class Author {

    @ManyToMany
    @JoinTable(
              name="AuthorBookGroup",
              joinColumns={@JoinColumn(name="fk_author",
referencedColumnName="id")},
              inverseJoinColumns={@JoinColumn(name="fk_group",
referencedColumnName="id")})
    @MapKey(name = "title")
    private Map<String, BookGroup> bookGroups = new HashMap<String,
BookGroup>();

    ...
}
```

That's all you need to do to model an association as a `Map` with multiple values per map key. You can now use it in the same way as I showed you in the first example.

Please be aware, that this mapping creates overhead and redundancy. The `BookGroup` entity gets mapped to a database table which we didn't need in the previous examples. And I now use the `title` of the `BookGroup` as the key of the `Map<String, BookGroup>`

`bookGroups`. The `BookGroup` `title` will be the same as the `title` of all `Book` s in the group.

Introducing JPA FetchTypes

The `FetchType` defines when Hibernate gets the related entities from the database, and it is one of the crucial elements for a fast persistence tier. In general, you want to fetch the entities you use in your business tier as efficiently as possible. But that's not that easy. You either get all relationships with one query or you fetch only the root entity and initialize the relationships as soon as you need them.

I'll explain both approaches in more detail in this chapter and also provide you some links to more advanced solutions that combine flexibility and efficiency.

Default FetchTypes and how to change them

When you started with Hibernate, you most, likely either didn't know about `FetchTypes` or you were told to always use `FetchType.LAZY`. In general, that's a good recommendation. But what does it exactly mean? And what is the default if you don't define the `FetchType`?

The default depends on the cardinality of the relationship. All to-one relationships use `FetchType.EAGER` and all to-many relationships `FetchType.LAZY`. I'll explain both of them in the following sections.

Even the best default doesn't fit for all use cases, and you sometimes want to change it. You can do this by providing your preferred `FetchType` to the relationship annotation as you can see in the following code snippet.

```
@Entity
@Table(name = "purchaseOrder")
public class Order implements Serializable {

  @OneToMany(mappedBy = "order", fetch = FetchType.EAGER)
  private Set<OrderItem> items = new HashSet<OrderItem>();

  ...

}
```

OK, now let's have a more detailed look at the different `FetchTypes`.

`FetchType.EAGER` - Fetch it so you'll have it when you need it

The `FetchType.EAGER` tells Hibernate to get all elements of a relationship when selecting the root entity. As I explained earlier, this is the default for to-one relationships, and you can see it in the following code snippets.

I use the default `FetchType` (`EAGER`) for the many-to-one relationship between the `OrderItem` and `Product` entity.

```
@Entity
public class OrderItem implements Serializable
{

   @ManyToOne
   private Product product;

   ...
}
```

When I now fetch an `OrderItem` entity from the database, Hibernate will also get the related `Product` entity.

```
OrderItem orderItem = em.find(OrderItem.class, 1L);
log.info("Fetched OrderItem: "+orderItem);
Assert.assertNotNull(orderItem.getProduct());
```

```
05:01:24,504 DEBUG SQL:92 - select orderitem0_.id as id1_0_0_, orderitem0_
.order_id as order_id4_0_0_, orderitem0_.product_id as product_5_0_0_,
orderitem0_.quantity as quantity2_0_0_, orderitem0_.version as
version3_0_0_, order1_.id as id1_2_1_, order1_.orderNumber as
orderNum2_2_1_, order1_.version as version3_2_1_, product2_.id as id1_1_2_,
product2_.name as name2_1_2_, product2_.price as price3_1_2_, product2_
.version as version4_1_2_ from OrderItem orderitem0_ left outer join
purchaseOrder order1_ on orderitem0_.order_id=order1_.id left outer join
Product product2_ on orderitem0_.product_id=product2_.id where orderitem0_
.id=?
05:01:24,557  INFO FetchTypes:77 - Fetched OrderItem: OrderItem , quantity:
100
```

This seems to be very useful in the beginning. Joining the required entities and getting all of them in one query is very efficient. But keep in mind, that Hibernate will ALWAYS fetch the `Product` entity for your `OrderItem`, even if you don't use it in your business code.

If the related entity isn't too big, this is not an issue for to-one relationships. But it will most likely slow down your application if you use it for a to-many relationship that you don't need for your use case. Hibernate then has to fetch tens or even hundreds of additional entities which creates a significant overhead.

`FetchType.LAZY` - Fetch it when you need it

The `FetchType.LAZY` tells Hibernate to only fetch the related entities from the database when you use the relationship. This is a good idea in general because there's no reason to select entities you don't need for your uses case. You can see an example of a lazily fetched relationship in the following code snippets.

The `one-to-many` relationship between the `Order` and the `OrderItem` entities uses the default `FetchType` for `to-many` relationships which is lazy.

```
@Entity
@Table(name = "purchaseOrder")
public class Order implements Serializable {

  @OneToMany(mappedBy = "order")
  private Set<OrderItem> items = new HashSet<OrderItem>();

  ...

}
```

The used `FetchType` has no influence on the business code. You can call the `getOrderItems()` method just as any other getter method.

```
Order newOrder = em.find(Order.class, 1L);
log.info("Fetched Order: "+newOrder);
Assert.assertEquals(2, newOrder.getItems().size());
```

Hibernate handles the lazy initialization transparently and fetches the `OrderItem` entities as soon as the getter method gets called.

```
05:03:01,504 DEBUG SQL:92 - select order0_.id as id1_2_0_, order0_
.orderNumber as orderNum2_2_0_, order0_.version as version3_2_0_ from
purchaseOrder order0_ where order0_.id=?
05:03:01,545  INFO FetchTypes:45 - Fetched Order: Order orderNumber: order1
05:03:01,549 DEBUG SQL:92 - select items0_.order_id as order_id4_0_0_,
items0_.id as id1_0_0_, items0_.id as id1_0_1_, items0_.order_id as
order_id4_0_1_, items0_.product_id as product_5_0_1_, items0_.quantity as
quantity2_0_1_, items0_.version as version3_0_1_, product1_.id as id1_1_2_,
product1_.name as name2_1_2_, product1_.price as price3_1_2_, product1_
.version as version4_1_2_ from OrderItem items0_ left outer join Product
product1_ on items0_.product_id=product1_.id where items0_.order_id=?
```

Handling lazy relationships in this way is perfectly fine if you work on a single Order entity or a small list of entities. But it becomes a performance problem when you do it on a large list of entities. As you can see in the following log messages, Hibernate has to perform an additional SQL statement for each `Order` entity to fetch its `OrderItems`.

```
05:03:40,936 DEBUG ConcurrentStatisticsImpl:411 - HHH000117: HQL: SELECT o
FROM Order o, time: 41ms, rows: 3
05:03:40,939  INFO FetchTypes:60 - Fetched all Orders
05:03:40,942 DEBUG SQL:92 - select items0_.order_id as order_id4_0_0_,
items0_.id as id1_0_0_, items0_.id as id1_0_1_, items0_.order_id as
order_id4_0_1_, items0_.product_id as product_5_0_1_, items0_.quantity as
quantity2_0_1_, items0_.version as version3_0_1_, product1_.id as id1_1_2_,
product1_.name as name2_1_2_, product1_.price as price3_1_2_, product1_
.version as version4_1_2_ from OrderItem items0_ left outer join Product
product1_ on items0_.product_id=product1_.id where items0_.order_id=?
05:03:40,957 DEBUG SQL:92 - select items0_.order_id as order_id4_0_0_,
items0_.id as id1_0_0_, items0_.id as id1_0_1_, items0_.order_id as
order_id4_0_1_, items0_.product_id as product_5_0_1_, items0_.quantity as
quantity2_0_1_, items0_.version as version3_0_1_, product1_.id as id1_1_2_,
product1_.name as name2_1_2_, product1_.price as price3_1_2_, product1_
.version as version4_1_2_ from OrderItem items0_ left outer join Product
product1_ on items0_.product_id=product1_.id where items0_.order_id=?
05:03:40,959 DEBUG SQL:92 - select items0_.order_id as order_id4_0_0_,
items0_.id as id1_0_0_, items0_.id as id1_0_1_, items0_.order_id as
order_id4_0_1_, items0_.product_id as product_5_0_1_, items0_.quantity as
quantity2_0_1_, items0_.version as version3_0_1_, product1_.id as id1_1_2_,
product1_.name as name2_1_2_, product1_.price as price3_1_2_, product1_
.version as version4_1_2_ from OrderItem items0_ left outer join Product
product1_ on items0_.product_id=product1_.id where items0_.order_id=?
```

This behavior is called n+1 select issue, and it's the most common performance problem. It is so common that you most likely have it if you didn't explicitly search for it. If you're not sure how to do that, please watch my 3-part video course about finding and fixing n+1 select issues [https://www.thoughts-on-java.org/library/video-training/]. It's available for free in the Thoughts on Java Library.

There are two ways to avoid these issues:

1. You can use `FetchType.EAGER` if you know that all of your use cases that fetch an `Order` entity also need to process the related `OrderItem` entities. That will almost never be the case.
2. If there are some use cases which only work on `Order` entities (which is most likely the case), you should use `FetchType.LAZY` in your entity mapping and use one of these options to initialize the relationship when you need them [https://www.thoughts-on-java.org/5-ways-to-initialize-lazy-relations-and-when-to-use-them/].

Thorben Janssen

Thorben Janssen has been a software developer and architect for more than 15 years. More than a decade ago, he used one of the first Hibernate releases to implement the persistence layer of enterprise applications. Since then, he has used the framework to implement applications of various sizes with complex business and performance requirements.

After blogging for several years about JPA and Hibernate, he decided to quit his day job in October 2016 to follow his passion for writing and teaching. Since then, he has been working as an independent trainer, author, and consultant to show software developers how to use Hibernate and JPA to avoid common problems and implement their persistence layer with ease.